Quantitative Analysis for the Mitigation of Multi-Stage Supply-Chain Attacks in Routine Automated CI/CD Workflows

A Formal-Methods Reconstruction of the Trivy / TeamPCP Compromise (CVE-2026-33634)

Franklin Hanna
[Affiliation] [email]

Abstract

Modern software delivery depends on Continuous Integration and Continuous Deployment (CI/CD) pipelines that automatically resolve and execute third-party code, GitHub Actions being the dominant example. When an attacker mutates a floating version tag of a widely used Action, thousands of downstream pipelines execute attacker-controlled code with production credentials on their next routine run. In March 2026 exactly this occurred: the trivy-action and setup-trivy Actions were compromised (CVE-2026-33634), leaking secrets that seeded a second-stage npm worm. We present a formal, quantitative analysis of this multi-stage attack. We encode the disclosed pre-attack state as a labelled transition system in TLA+ and exhaustively check, with the TLC model checker, whether the documented attack is reachable and which mitigations provably close it. We prove a per-pipeline isolation property and establish a refinement relation between a hardened workflow and an abstract secure specification, quantifying the residual attack surface. We then lift the model into a Markov Decision Process and use the PRISM probabilistic model checker to compute exact compromise probabilities, expected time-to-compromise, and, via a two-stage cascade model, downstream propagation, including closed-form parametric results. The analysis is validated in three layers: structural faithfulness against a corpus of 189 real workflows, incident reconstruction against the public dossier, and predictive calibration against the OpenSSF/OSV malicious-package dataset. Our central finding is that the model formally distinguishes complete credential rotation, which prevents the attack, from the partial rotation that actually occurred, which does not, matching the documented incident causation, and that universal commit-SHA pinning isolates a pipeline even when the stolen credential remains valid.

Keywords: software supply-chain security; CI/CD; GitHub Actions; formal methods; model checking; TLA+; probabilistic model checking; PRISM; refinement; isolation.

1 Introduction

A continuous-integration pipeline is, operationally, a program that fetches and executes code chosen by a version reference. The overwhelmingly common reference in GitHub Actions is a floating tag such as `actions/checkout@v4`: a mutable pointer whose target commit the repository owner can change at any time. This convenience is also a capability. An adversary who can move a tag can, without touching any victim, cause every pipeline that resolves that tag to execute attacker-controlled code on its next routine run, with whatever secrets that runner holds.

In March 2026 the trivy-action and setup-trivy GitHub Actions, maintained by Aqua Security, were compromised in precisely this way (CVE-2026-33634) [1, 3]. A credential stolen in a February incident was left valid; in March the attacker used it to force-push 76 of 77 tags on trivy-action and all 7 tags on setup-trivy to malicious commits [7]. Victim runners executed the payload, exfiltrated cloud credentials, SSH keys, Kubernetes configurations and CI secrets [8], and the stolen npm tokens seeded a second-stage worm (CanisterWorm) that propagated through the npm ecosystem [6].

Public post-incident analyses are thorough but qualitative: they narrate what happened. They do not answer the questions a defender must answer before an incident: which mitigations provably eliminate the attack rather than merely inconveniencing it, whether a partial remediation is sufficient, and how quickly and how far a compromise is expected to spread. These are formal and quantitative questions. This paper answers them for the Trivy incident by construction, and offers the construction as a reusable template.

Contributions. We make the following contributions:

- A TLA+ transition-system model of the Trivy/TeamPCP pre-attack state whose every constant and variable traces to a cited dossier claim, and an exhaustive TLC proof that the documented attack is reachable from that state and is closed by (i) complete credential rotation and (ii) universal SHA-pinning.
- A formal per-pipeline *isolation* theorem: in a mixed population, a SHA-pinned pipeline is provably never compromised even while its tag-pinned neighbours are, so the blast radius is contained to the unpinned subset.

- A *refinement* relation between a hardened workflow and an abstract secure specification, with the residual attack surface quantified as exactly the set of unpinned victims.
- A PRISM Markov model computing exact compromise probability, expected time-to-compromise, a two-stage npm-propagation cascade, and closed-form parametric results, calibrated against real malicious-package frequency data.
- A three-layer validation methodology (structural faithfulness, incident reconstruction, predictive calibration) that makes the model's fidelity auditable.

2 Background: The Trivy / TeamPCP Compromise

We summarise the incident from the public dossier [1]–[13]. In February 2026 an Aqua Security continuous-integration service account was compromised. The February response rotated some credentials but left the service-account credential valid, a partial remediation. In March 2026 the attacker, holding that residual credential with tag-write permission on the Action repositories, force-pushed the repositories' version tags to point at malicious commits. Because the vast majority of downstream consumers reference the Actions by floating tag rather than by pinned commit SHA, the next scheduled or triggered run of each victim pipeline resolved the tag to the malicious commit and executed it in the runner context.

The payload read the runner environment and exfiltrated its secrets to attacker-controlled infrastructure. Among the stolen secrets were npm publish tokens, which enabled a distinct second stage: publication of malicious npm packages (the CanisterWorm family) that propagated to downstream package consumers [6, 9]. The campaign's first publicly named victim, Mercor, attributes its breach to a follow-on compromise in the same campaign (the LiteLLM library) rather than to direct execution of a malicious Action tag [5] — a measure of how far downstream the cascade reached. The incident is thus genuinely multi-stage: a credential-residual stage enabling a tag-mutation stage enabling a secret-exfiltration stage enabling an ecosystem-propagation stage.

3 Threat and System Model

We fix the boundary of formal claims before modelling. In scope are: the Aqua service-account credential and its rotation state; the Action repositories and their tag-to-commit mappings; victim runners and the secrets present in their environment at execution time; the attacker's accumulated knowledge (exfiltrated secrets); each victim's pinning policy (tag versus SHA); and the completeness of credential rotation.

Deliberately abstracted, and argued not to affect the reachability property under study, are: how the attacker first obtained the credential (taken as an initial condition); the command-and-control network path (exfiltration is modelled as set union into the attacker's knowledge); victim production systems downstream of the stolen secrets; Git object-storage internals (tag mutation is one atomic action); the platform's internal authentication and authorization; and the byte-level structure of the payload (only its active behaviour, transferring secrets, is modelled).

The adversary begins holding solely the residual Aqua service-account credential. Its capability is tag mutation on the Action repositories, conditioned on that credential remaining valid. Victim pipelines are honest but automated: each runs its workflow on its own schedule, resolving the Action reference according to its pinning policy.

4 Formal Framework

4.1 Transition-system model

We model the system as a labelled transition system in TLA+ [16]. The state comprises the current tag-to-commit map, each victim's pinning policy, the service-account rotation state, the credentials the attacker controls, the attacker's exfiltrated-secret set, each runner's secrets, and each SHA-pinning victim's locked commit. Two actions generate all behaviour. `ForcePushTag` repoints a tag to the malicious commit, guarded by the attacker holding a still-valid credential. `RunnerExecute` fires when a victim runs the Action: if the commit it resolves is malicious, that victim's secrets union into the attacker's knowledge. Listing 1 gives the two actions.

The safety property is `NoExfiltration`: the attacker's knowledge never intersects any victim's secrets. TLC checks it exhaustively over a model with five victims, five tags, three commits and two repositories, a scale more than sufficient to exercise the reachability claim while remaining trivially checkable.

4.2 Reachability and mitigation discrimination

On the vulnerable configuration TLC returns the documented attack as a shortest counterexample (Figure 1), confirming the formalism can represent the incident. Two mitigation knobs then parameterise the initial state: whether the credential was fully rotated, and which victims pin by SHA. Checking `NoExfiltration` across the four extreme configurations yields the discrimination in Table 2.

4.3 Isolation between pipelines

Generalising the SHA-pinning knob from a Boolean to a set of victims lets us reason about mixed populations. Define `Isolation` to hold when no SHA-pinning victim's secret is ever exfiltrated. Checking it on a mixed configuration, in which two victims pin by SHA and three use floating tags with the credential still valid, TLC reports that the invariant holds over the entire reachable state space, even though a companion check confirms the attack does compromise the tag-pinned victims in the very same runs. A hardened pipeline is therefore isolated from the blast radius of its unpinned neighbours; the breach is provably contained to the unpinned subset.

Listing 1: The two TLA+ actions. A tag-pinned victim resolves the (possibly mutated) tag; a SHA-pinned victim resolves its audited commit and is immune.

```
ForcePushTag(r, t) ==
  /\ AttackerCanForcePush              \* holds a still-valid credential
  /\ tagMap[<<r, t>>] # MaliciousCommit
  /\ tagMap' = [tagMap EXCEPT ![<<r,t>>] = MaliciousCommit]
  /\ UNCHANGED <<pinningPolicy, rotationState, ...>>

RunnerExecute(v, r, t) ==
  /\ ExecutedCommit(v, r, t) = MaliciousCommit
  /\ attackerKnowledge' = attackerKnowledge \cup victimSecrets[v]
  /\ UNCHANGED <<tagMap, pinningPolicy, ...>>
```

```
State 1  [Init]        attacker holds AquaServiceAcct; all tags -> c_benign
State 2  ForcePushTag(trivy_action, tg1)          tag -> c_malicious
State 3  RunnerExecute(v1, trivy_action, tg1)     attackerKnowledge = {v1}
```

Figure 1: TLC counterexample on the vulnerable configuration, structurally identical to the documented dossier trace.

4.4 Refinement and residual attack surface

We specify an abstract secure workflow exposing a single observation, `leaked`, whose only permitted transitions leave it false. Under the refinement mapping from `leaked` to the predicate that a victim secret has reached the attacker, the hardened concrete workflow refines the abstract specification (TLC verifies the refinement holds), whereas the vulnerable workflow does not: TLC exhibits a behaviour whose mapped observation flips from false to true, namely the documented attack. The set of such flipping transitions is the residual attack surface. TLC witnesses that, for the vulnerable configuration, the reachable compromised set equals exactly the unpinned victims; growing the SHA-pinned set shrinks this surface monotonically to the empty set.

4.5 Probabilistic and multi-stage model

Reachability answers *whether*; it does not answer how likely, how fast, or how far. We lift the two actions into a Markov model checked with PRISM [17]. The attacker's weaponisation timing is nondeterministic (an adversary, resolved by PRISM's schedulers); the victim's build cadence is probabilistic with per-day probability p. Maximum reachability probability and minimum expected reward then give worst-case (most-capable-adversary) quantities. A separate discrete-time cascade model chains the two documented stages: stage one yields a stolen npm token with probability q, enabling stage two, in which downstream consumers adopt the malicious npm package at per-day rate r until a population of size N is saturated.

4.6 Parametric analysis

Because the input rates are uncertain, we treat them symbolically. PRISM's parametric engine returns closed-form functions rather than point estimates: the expected time-to-compromise is $(p+1)/p$, and the probability of reaching the second stage is exactly q. Such closed forms make the dependence on each uncertain input explicit and support the sensitivity argument of Section 6.

5 Three-Layer Validation Methodology

A formal model is only as valuable as its faithfulness to reality. We validate in three layers. Layer 1 (structural faithfulness) asks whether the model captures what real workflows do, by statically analysing a corpus of real GitHub Actions workflows and measuring the frequency of the constructs the model represents. Layer 2 (incident reconstruction) asks whether the model can reproduce the documented attack and whether its proposed mitigations close it, checking the encoded pre-attack state against the public dossier. Layer 3 (predictive calibration) asks whether the model's quantitative outputs are consistent with observed frequencies, calibrating input distributions against public malicious-package data and the documented incident timeline.

6 Evaluation

6.1 Layer 1: structural faithfulness

We analysed 189 workflow files drawn from eighteen large open-source projects, extracting trigger events, permission scopes, secret usage, action-version pinning, runner types, job dependencies and artefact flows. The headline calibration parameter is the fraction of external Action references that use a floating tag rather than a pinned commit SHA (Table 1).

The measured floating-tag fraction feeds Layer 3 directly. Because the corpus consists of large, security-mature projects that pin more than average, the 37% figure is a lower bound on the ecosystem-wide rate; the model's safety proof instead uses the worst case of universal tag reference, the conservative choice for a safety claim. The model captures 88.2% of the compromise-relevant constructs observed; the remainder (matrix strategies, artefact flows, reusable-workflow nesting)

Table 1: Layer 1 corpus measurements (189 workflows, 18 projects, 0 parse errors).

Quantity	Value
External (resolvable) Action references	1,517
Commit-SHA pinned	956 (63.0%)
Floating tag / branch (unpinned)	561 (37.0%)
Floating-tag fraction f	0.3698
Construct coverage of the model	88.2%
Workflows using `pull_request_target`	11.6%
Workflows referencing secrets	50.3%

are independent of the tag-mutation reachability property.

6.2 Layer 2: incident reconstruction

Each row of Table 2 is a separate TLC run with a different initial state; the vulnerable row reproduces the documented attack and the mitigated rows show which remediations close it. Table 3 records the isolation and refinement results of Sections 4.3–4.4.

The load-bearing result is the contrast between the first two rows of Table 2: the model formally distinguishes complete rotation (safe) from the partial rotation that actually occurred (still exploited), reproducing the documented incident causation. The first and third rows give the second, independent finding: universal SHA-pinning protects victims even while the stolen credential remains valid.

6.3 Layer 3: quantitative outputs

With build cadence $p = 0.2$ per day and a 30-day horizon, PRISM yields the quantitative counterpart of Table 2 (Table 4).

The expected time of six days decomposes exactly as one day to weaponise plus $1/p = 5$ days to the next build. A sensitivity sweep of p across a tenfold range (0.05 to 0.5 per day) moves the vulnerable expected time from 21 to 3 days, but SHA-pinning yields probability zero at every value of p: the mitigation ranking is invariant under a tenfold change in the compromise rate, so the recommendation is robust to parameter uncertainty.

6.4 Multi-stage propagation

The cascade model quantifies the second stage. With $p = 0.2$, token-theft probability $q = 0.5$, adoption rate $r = 0.1$ and downstream population $N = 5$, PRISM gives Table 5.

Complete credential rotation drives the probability of the entire downstream cascade, not merely the first stage, to zero. The expected sixteen days to the first downstream compromise decomposes as one day to weaponise, five to first-stage compromise, and ten to first downstream adoption.

6.5 Calibration against public data

We calibrate the second-stage rates against the OpenSSF malicious-packages dataset [15], the OSV-format corpus seeded by the Backstabber's Knife Collection of Ohm et al. [14]. Over the dataset, npm accounts for 214,497 malicious-package reports, 94.2% of all ecosystems. Crucially, npm malicious-package publications rose from 329 in February 2026 to 1,048 in March 2026, a factor of 3.19, in the exact month of the CanisterWorm second stage; the Trivy-derived package was one of those 1,048. The model's fastest-adversary times (six days to first-stage compromise, sixteen to first downstream) are lower bounds consistent with the documented four-week February-to-March residual-credential window; the surplus reflects adversarial patience, which the nondeterministic weaponisation timing of the MDP captures exactly.

6.6 Mitigation discovery

Finally, we use the verified model as a ground-truth oracle for an automated search over defender policies, scoring each candidate by its model-checked compromise probability plus an operational-cost penalty [20]. Maximising the score converges on complete credential rotation as the minimal-cost provably-safe policy, recovering the paper's central recommendation without human guidance. The search proposes; the model checker decides.

7 Discussion

Two findings are non-trivial in the precise sense that they contradict a plausible operational intuition. First, a partial credential rotation that revokes most credentials is not a partial defence but no defence at all against this attack class: a single retained valid credential with tag-write permission suffices, and the model flags exactly this. Second, commit-SHA pinning defends a pipeline unilaterally. A pinning victim is protected regardless of its neighbours' behaviour and regardless of whether the upstream credential is ever rotated, because it never resolves the mutated tag. Isolation formalises this as containment of the blast radius, and refinement quantifies the residual surface as precisely the set of pipelines that decline to pin.

The quantitative layer sharpens the guidance rather than replacing it. Expected-time and bounded-horizon probabilities let an operator reason about detection windows; the parametric forms make explicit which conclusions are robust (the mitigation ranking) and which depend on uncertain inputs (absolute timings). The multi-stage model shows that the value of a mitigation is not confined to the pipeline that adopts it: preventing the first stage removes the token supply that powers the second.

8 Limitations and Threats to Validity

The analysis is deliberately conservative, and several caveats bound its claims. On data: the second-stage parameters q (the fraction of compromised runners holding npm tokens) and r (downstream adoption rate) are assumptions rather than measurements, for want of a public dataset; we therefore report the parametric form $P(\text{stage } 2) = q$ so the dependence is explicit. The workflow corpus is modest (189 files) and

Table 2: Mitigation discrimination under exhaustive model checking. Each row is one TLC run.

Configuration	`NoExfiltration`	Matches reality
Tag refs + valid stolen credential (actual)	FAILS	Yes – attack occurred
Tag refs + complete rotation	holds	No attack
SHA pins + valid stolen credential	holds	Protects despite residual credential
SHA pins + complete rotation	holds	Full defence

Table 3: Isolation and refinement results (TLC).

Property checked	Configuration	Result
Isolation (SHA-pinned never leak)	mixed population	holds (8,185 states)
Containment witness	mixed population	breach reaches only unpinned victims
Hardened refines `SecureWorkflow`	all SHA-pinned	holds
Vulnerable refines `SecureWorkflow`	all floating tags	fails (observation flips)
Residual attack surface	all floating tags	= unpinned subset (all 5)

Table 4: Quantitative mitigation table (PRISM MDP; $p = 0.2$/day, horizon 30 days).

Configuration	$P(\text{comp.})$	$E[\text{days}]$	$P(\leq 30\text{d})$
Tag + valid credential (actual)	1.00	6.00	0.9985
Tag + complete rotation	0.00	∞	0.0000
SHA + valid credential	0.00	∞	0.0000
SHA + complete rotation	0.00	∞	0.0000

Table 5: Two-stage propagation (PRISM DTMC).

Metric	Value
$P(\text{reach stage 2})$	0.50 ($= q$)
$P(\text{stage 2 within 30 days})$	0.454
$P(\text{full downstream propagation})$	0.50
$E[\text{hosts compromised at day 30}]$	2.18
$E[\text{days to first downstream}]$ (q=1)	16.0
$P(\text{reach stage 2})$ under rotation	0.00

popularity-biased toward projects that pin more than average, so the measured floating-tag fraction is a lower bound. The build cadence p is illustrative. The calibration against the February-to-March publication spike is corroborative, not causal, and the incident facts themselves are taken from the public dossier rather than independently re-verified.

On modelling: the checked instance is small (five victims), sufficient to exercise the safety property but not a proof for arbitrary populations; several mechanisms are abstracted by design (the credential-theft origin, the C2 path, Git internals, payload structure); the second-stage population dynamics are simplified to sequential adoption; and the worst-case adversary means the reported probabilities and times are bounds rather than expected real-world values. On methodology and tooling: PRISM is used in place of Storm [19] (identical semantics, no native Windows build for the latter); the parametric engine does not support step-bounded properties, for which we give the analytic form; the automated policy search's outer loop was not executed end-to-end; a single incident is reconstructed where the methodology envisions several; and a practitioner review of the threat model, which the methodology recommends, has not been conducted.

9 Related Work

Empirical study of open-source supply-chain attacks was catalogued by Ohm et al. in the Backstabber's Knife Collection [14], and is tracked continuously by the OpenSSF malicious-packages project and OSV [15] and by industry telemetry [20]. Defensive frameworks such as SLSA [18] prescribe provenance and pinning controls; our contribution is to give one such control (SHA-pinning) a formal isolation guarantee and a quantified residual surface rather than a checklist status. Formal verification via TLA+ and TLC [16] and probabilistic model checking via PRISM [17] and Storm [19] are mature; we apply them jointly to a documented CI/CD incident and, unusually, validate the abstraction against a real workflow corpus and against measured malicious-package frequencies rather than treating the model as self-justifying.

10 Conclusion

We reconstructed the March 2026 Trivy/TeamPCP supply-chain compromise as a formal, quantitative artefact. Exhaustive model checking shows the documented attack is reachable from the disclosed pre-attack state and is closed by complete credential rotation or by universal SHA-pinning; an isolation theorem and a refinement relation make the SHA-pinning guarantee precise and quantify the residual attack surface; and a probabilistic, multi-stage, parametric model, calibrated against public data, quantifies how likely, how fast,

and how far a compromise spreads. The recurring lesson is that in automated CI/CD the difference between a partial and a complete remediation is not a difference of degree but of kind, and that a formalism which distinguishes them says something an incident report cannot.

References

[1] Aqua Security. Trivy supply-chain attack: what you need to know. Technical report, 2026.

[2] GitHub. Security Advisory for aquasecurity/trivy. GitHub Security Advisories, 2026.

[3] MITRE. CVE-2026-33634. Common Vulnerabilities and Exposures, 2026.

[4] CISA. Known Exploited Vulnerabilities Catalog, 2026.

[5] Mercor. Public disclosure statement on the TeamPCP supply-chain campaign (LiteLLM follow-on compromise), 2026.

[6] Palo Alto Networks, Unit 42. TeamPCP supply-chain attacks and the CanisterWorm payload, 2026.

[7] Microsoft. Detecting, investigating and defending against the Trivy supply-chain compromise. Microsoft Security Blog, 24 March 2026.

[8] Wiz Research. Trivy compromised: tracking the TeamPCP supply-chain attack, 2026.

[9] ReversingLabs. The TeamPCP supply-chain attack spreads: payload analysis, 2026.

[10] Endor Labs. TeamPCP isn't done: the February-to-March causal chain, 2026.

[11] Kudelski Security. Investigating two variants of the Trivy supply-chain compromise, 2026.

[12] SANS Internet Storm Center. Diary entry 32856: Trivy Action compromise, 2026.

[13] Cato Networks. The TeamPCP supply-chain attack, 2026.

[14] M. Ohm, H. Plate, A. Sykosch, and M. Meier. Backstabber's Knife Collection: A Review of Open-Source Software Supply-Chain Attacks. In *DIMVA*, 2020.

[15] Open Source Security Foundation. Malicious Packages repository (OSV format), 2026.

[16] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[17] M. Kwiatkowska, G. Norman, and D. Parker. PRISM 4.0: Verification of Probabilistic Real-Time Systems. In *CAV*, 2011.

[18] Open Source Security Foundation. SLSA: Supply-chain Levels for Software Artifacts.

[19] C. Hensel, S. Junges, J.-P. Katoen, T. Quatmann, and M. Volk. The Probabilistic Model Checker Storm. *STTT*, 2022.

[20] Sonatype. State of the Software Supply Chain / Open-Source Malware Index. Industry report, 2025.